Università degli Studi di Palermo

dipartimento di ingegneria unipa

# Introduzione a numpy

INTELLIGENZA ARTIFICIALE PER L'INGEGNERIA BIOMEDICA (EX ROBOTICA MEDICA)

a.a. 2025/2026

Prof. Roberto Pirrone

CHILAB

# Cos'è numpy e come installarlo

Il pacchetto fondamentale per il calcolo numerico in Python

## Cos'è numpy?

- Libreria open-source per il calcolo numerico ad alte prestazioni in Python
- Fornisce l'oggetto ndarray: array n-dimensionale omogeneo e compatto in memoria
- Operazioni vettorializzate eseguite in C/Fortran: molto più veloci dei loop Python
- Base di SciPy, Pandas, scikit-learn, TensorFlow e molte altre librerie

## Installazione

**Con pip (tutti i sistemi):**

Terminale

```
pip install numpy

# aggiornare una versione esistente:
pip install --upgrade numpy
```

**Con conda / Anaconda:**

Terminale

```
conda install numpy

# verifica versione installata:
python -c "import numpy; print(numpy.__version__)"
```

**Convenzione**

Importazione convenzionale: `import numpy as np`

Tutti gli esempi di questa presentazione assumono che numpy sia importato come np.

# Creazione di array da iterabili

np.array() e funzioni di utilità

## Da lista o tupla Python

Python

```python
import numpy as np

# da lista
a = np.array([1, 2, 3, 4, 5])
print(a)          # [1 2 3 4 5]

# da tupla
b = np.array((0.5, 1.5, 2.5))
print(b.dtype)    # float64

# dtype esplicito
c = np.array([1, 2, 3], dtype=np.float32)
```

## Funzioni di generazione

```python
np.zeros(5)           # [0. 0. 0. 0. 0.]
np.ones(4)            # [1. 1. 1. 1.]
np.full(3, 7)         # [7 7 7]
np.eye(3)             # matrice identità 3×3
```

## Sequenze numeriche

Python

```python
# arange: simile a range()
np.arange(0, 10, 2)   # [0 2 4 6 8]
np.arange(5)          # [0 1 2 3 4]

# linspace: n punti equispaziati
np.linspace(0, 1, 5)  # [0.   0.25 0.5  0.75 1. ]

# logspace: punti equispaziati in scala log
np.logspace(0, 2, 3)  # [  1.  10. 100.]
```

## Da altri iterabili

```python
# da generatore
gen = (x**2 for x in range(5))
np.fromiter(gen, dtype=int)  # [0 1 4 9 16]

# da stringa / file
np.fromstring('1 2 3', sep=' ')  # [1. 2. 3.]
```

# Array bidimensionali

Matrici e operazioni di base

## Creazione di matrici

**Python**

```python
# lista di liste → matrice 3×4
M = np.array([[1, 2, 3, 4],
              [5, 6, 7, 8],
              [9,10,11,12]])

print(M.shape)   # (3, 4)
print(M.ndim)    # 2
print(M.size)    # 12

# matrici speciali
np.zeros((3, 4))     # tutti zeri
np.ones((2, 5))      # tutti uno
np.eye(4)            # matrice identità 4×4
np.diag([1, 2, 3])   # matrice diagonale
```

## Accesso agli elementi

**Python**

```python
M = np.array([[1,2,3],[4,5,6],[7,8,9]])

M[0, 1]      # riga 0, col 1  →  2
M[2, :]      # terza riga     →  [7 8 9]
M[:, 1]      # seconda colonna→  [2 5 8]
M[0:2, 1:3]  # sotto-matrice  →  [[2 3],[5 6]]
```

## Operazioni di base

```python
A = np.array([[1,2],[3,4]])
B = np.array([[5,6],[7,8]])

A + B          # somma elemento per elemento
A * B          # prodotto elemento per elemento
A @ B          # prodotto matriciale (matmul)
np.dot(A, B)   # equivalente a A @ B

A.sum()        # somma di tutti gli elementi
A.sum(axis=0)  # somma per colonne  → [4 6]
A.sum(axis=1)  # somma per righe    → [3 7]
A.mean(), A.std(), A.min(), A.max()
```

# Shape, dtype, reshape e trasposta

Attributi fondamentali e riorganizzazione dei dati

## Attributi shape e dtype

**Python**

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

a.shape    # (2, 3)  →  2 righe, 3 colonne
a.dtype    # int64   (o int32 su Windows)
a.ndim     # 2       →  numero di assi
a.size     # 6       →  numero totale di elementi
a.itemsize # 8       →  byte per elemento

# cambio di tipo
a.astype(np.float64)  # copia con dtype float64
a.astype(np.complex128)
```

## Reshape

```
a = np.arange(12)       # [0..11]  shape (12,)

b = a.reshape(3, 4)     # shape (3, 4)
c = a.reshape(2, 2, 3)  # shape (2, 2, 3)

# -1: numpy calcola la dimensione automaticamente
d = a.reshape(4, -1)    # shape (4, 3)
e = a.reshape(-1, 6)    # shape (2, 6)

# flatten: torna sempre a 1D (copia)
b.flatten()             # shape (12,)
# ravel: come flatten ma evita copia se possibile
b.ravel()               # shape (12,)
```

## Trasposta

**Python**

```
M = np.array([[1, 2, 3],
              [4, 5, 6]])   # shape (2, 3)

T = M.T                     # shape (3, 2)
# equivalente:
T = M.transpose()           # shape (3, 2)
T = np.transpose(M)         # shape (3, 2)

# per array n-dimensionali: specificare gli assi
A = np.ones((2, 3, 4))
A.transpose(2, 0, 1).shape  # (4, 2, 3)
```

**View vs copia**

reshape() restituisce una view (non una copia) quando possibile:
modifiche alla view si riflettono sull'array originale.
Usa .copy() per ottenere un array indipendente.

## Aggiungere / rimuovere assi

```
v = np.array([1, 2, 3])       # shape (3,)
v[:, np.newaxis]              # shape (3, 1)  colonna
v[np.newaxis, :]              # shape (1, 3)  riga
np.expand_dims(v, axis=0)     # shape (1, 3)
np.squeeze(v[:, np.newaxis])  # shape (3,)
```

# Notazione slice e maschere booleane

Selezione e filtraggio degli elementi di un array

## Slice su array 1D

Python

```
a = np.arange(10)   # [0 1 2 3 4 5 6 7 8 9]

a[2:5]     # [2 3 4]
a[:4]      # [0 1 2 3]
a[6:]      # [6 7 8 9]
a[::2]     # [0 2 4 6 8]   passo 2
a[::-1]    # [9 8 7 6 5 4 3 2 1 0]  inverso
a[1:8:3]   # [1 4 7]
```

## Slice su array 2D

```
M = np.arange(16).reshape(4, 4)

M[1, :]      # seconda riga intera
M[:, 2]      # terza colonna intera
M[0:2, 1:3]  # sotto-matrice righe 0-1, col 1-2
M[::2, ::2]  # righe e colonne pari

# assegnazione via slice (modifica in-place)
M[0, :] = 0  # azzera la prima riga
M[:, 0] = -1 # prima colonna a -1
```

## Maschere booleane (boolean indexing)

Python

```
a = np.array([3, -1, 4, -1, 5, -9, 2, 6])

# creare la maschera
mask = a > 0
# mask: [ True False  True False  True False  True  True]

# selezionare gli elementi
a[mask]          # [3 4 5 2 6]
a[a > 0]         # forma compatta equivalente

# modificare gli elementi selezionati
a[a < 0] = 0     # azzera i negativi
# a: [3 0 4 0 5 0 2 6]
```

## Maschere con np.where e operatori logici

```
a = np.arange(-3, 4)   # [-3 -2 -1  0  1  2  3]

# np.where(cond, val_true, val_false)
np.where(a >= 0, a, -a)  # valore assoluto

# combinare condizioni
(a > -2) & (a < 2)   # [-1  0  1]
(a < -1) | (a > 1)   # [-3 -2  2  3]
~(a == 0)             # tutti tranne 0

# indici degli elementi veri
np.where(a > 0)       # (array([4, 5, 6]),)
np.nonzero(a > 0)     # equivalente
```

# Broadcasting – Concetto e regole

Come numpy gestisce operazioni tra array di shape diverse

## Cos'è il broadcasting?

Il broadcasting permette di applicare operazioni tra array di shape diverse senza creare copie esplicite dei dati.

## Regole (applicate da destra a sinistra sulle shape):

**Regola 1**
Se i due array hanno numero diverso di dimensioni, la shape di quello con meno dimensioni viene prefissata con 1.

**Regola 2**
Le dimensioni di size 1 vengono 'estese' alla dimensione dell'altro array (senza copiare dati).

**Regola 3**
Se le dimensioni non sono né uguali né pari a 1 in almeno un array, si genera un ValueError.

## Esempi di shape compatibili

`(3,) + scalar` → `(3,)` aggiunge lo scalare a ogni elemento

`(3, 4) + (4,)` → `(3, 4)` riga sommata a ogni riga della matrice

`(3, 1) + (1, 4)` → `(3, 4)` prodotto esterno (vedi slide successiva)

`(2,3,4)+ ( 4)` → `(2,3,4)` estensione sull'ultimo asse

**■ Efficienza**

Le operazioni di broadcasting non allocano memoria extra per gli array estesi: numpy usa strides per simulare la ripetizione in modo efficiente.

# Broadcasting – Espressioni, ufunc e prodotto esterno

Operazioni vettorizzate e esempio pratico

## Espressioni aritmetiche e ufunc

**Python**

```
a = np.array([1., 2., 3., 4.])
b = np.array([10., 20., 30., 40.])

# operazioni elemento per elemento
a + b      # [11. 22. 33. 44.]
a * b      # [10. 40. 90. 160.]
a ** 2     # [ 1.  4.  9. 16.]   broadcasting scalare
b / a      # [10. 10. 10. 10.]

# ufunc: funzioni universali applicate element-wise
np.sqrt(a)          # [1.    1.414 1.732 2.   ]
np.exp(a)           # [e  e^2  e^3  e^4]
np.sin(np.pi * a)   # quasi zero per interi
np.log(b)           # [2.30 3.00 3.40 3.69]

# aggregazioni con asse
M = np.arange(6).reshape(2,3)
M - M.mean(axis=1, keepdims=True)  # normaliz. righe
```

## Prodotto esterno di due vettori

Il prodotto esterno di u (m,) e v (n,) produce una matrice (m, n):

**M[i,j] = u[i] * v[j]**

**Python**

```
u = np.array([1, 2, 3])        # shape (3,)
v = np.array([10, 20, 30, 40]) # shape (4,)

# Metodo 1: np.outer()
M = np.outer(u, v)

# Metodo 2: broadcasting con newaxis
M = u[:, np.newaxis] * v[np.newaxis, :]
#   u[:, None] ha shape (3, 1)
#   v[None, :] ha shape (1, 4)
#   risultato:   shape (3, 4)
```

**Output**

```
M = [[ 10  20  30  40]
     [ 20  40  60  80]
     [ 30  60  90 120]]
shape: (3, 4)
```

**■ Generalizzazione**

np.multiply.outer(u, v) generalizza il prodotto esterno
a qualsiasi ufunc binaria (es. np.add.outer, np.subtract.outer).

# numpy.linalg – Autovalori e sistemi lineari

Algebra lineare con np.linalg

## Autovalori e autovettori

Per $A \cdot v = \lambda \cdot v$ → np.linalg.eig(A) restituisce ($\lambda$, V)

**Python**

```
A = np.array([[4., 2.],
              [1., 3.]])

# autovalori e autovettori (matrice generica)
eigenvalues, eigenvectors = np.linalg.eig(A)

print(eigenvalues)   # [5. 2.]
print(eigenvectors)  # colonne = autovettori
# [[ 0.894 -0.707]
#  [ 0.447  0.707]]

# verifica: A @ v ≈ λ * v
v0 = eigenvectors[:, 0]      # primo autovettore
np.allclose(A @ v0, eigenvalues[0] * v0)  # True

# per matrici simmetriche/hermitiane (più stabile)
np.linalg.eigh(A)   # garantisce autoval. reali
```

## Risoluzione di sistemi lineari Ax = b

**Python**

```
# Sistema:  2x + y = 5
#            x + 3y = 10
A = np.array([[2., 1.],
              [1., 3.]])
b = np.array([5., 10.])

# soluzione esatta (usa fattorizzazione LU)
x = np.linalg.solve(A, b)
print(x)  # [1. 3.]

# verifica residuo
np.allclose(A @ x, b)  # True
```

## Altre funzioni utili di linalg

```
np.linalg.det(A)      # determinante
np.linalg.inv(A)      # matrice inversa
np.linalg.matrix_rank(A)  # rango

# norme
np.linalg.norm(b)          # norma L2 (Euclidea)
np.linalg.norm(b, ord=1)   # norma L1
np.linalg.norm(A, ord='fro')  # norma di Frobenius

# decomposizioni
U, s, Vt = np.linalg.svd(A)   # SVD
Q, R     = np.linalg.qr(A)    # QR
L        = np.linalg.cholesky(A.T @ A)  # Cholesky

# minimi quadrati (sistema sovradeterminato)
x, res, rank, sv = np.linalg.lstsq(A, b, rcond=None)
```

# Riepilogo e Risorse

numpy in una pagina

## Concetti chiave visti

- **Installazione:** `pip install numpy / conda install numpy`
- **Array da iterabili:** `np.array(), np.arange(), np.linspace(), np.fromiter()`
- **Array 2D:** `lista di liste, np.zeros/ones/eye, operatori + * @`
- **Shape e dtype:** `a.shape, a.dtype, a.reshape(), a.T, np.newaxis`
- **Slice:** `a[start:stop:step], M[row, col], M[r1:r2, c1:c2]`
- **Maschere booleane:** `mask = a > 0; a[mask]; np.where(cond, x, y)`
- **Broadcasting:** `operazioni tra shape diverse; regole destra→sinistra`
- **Prodotto esterno:** `np.outer(u,v) ≡ u[:,None] * v[None,:]`
- **linalg.eig:** `autovalori/vettori: np.linalg.eig(A)`
- **linalg.solve:** `sistemi lineari: np.linalg.solve(A, b)`

## Risorse ufficiali

- **Documentazione numpy:** https://numpy.org/doc/stable/
- **Quickstart tutorial:** https://numpy.org/doc/stable/user/quickstart.html
- **Guida assoluti principianti:** https://numpy.org/doc/stable/user/absolute_beginners.html
- **Broadcasting spiegato:** https://numpy.org/doc/stable/user/basics.broadcasting.html
- **numpy.linalg reference:** https://numpy.org/doc/stable/reference/routines.linalg.html
- **100 numpy Exercises:** https://github.com/rougier/numpy-100

## Prossimi passi consigliati

→ SciPy: estensioni scientifiche (integrazione, ottimiz., segnali)

→ Matplotlib: visualizzazione di array e matrici

→ Pandas: dataframe costruiti su numpy array

→ scikit-learn: machine learning con API numpy-compatibile